GOSIM

GOSIM Paris 2026

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris

GOSIM

GOSIM Paris 2026

Robotics with Eclipse Zenoh

From ROS 2 to ros-z and beyond

Julien Enoch - ZettaScale Technology

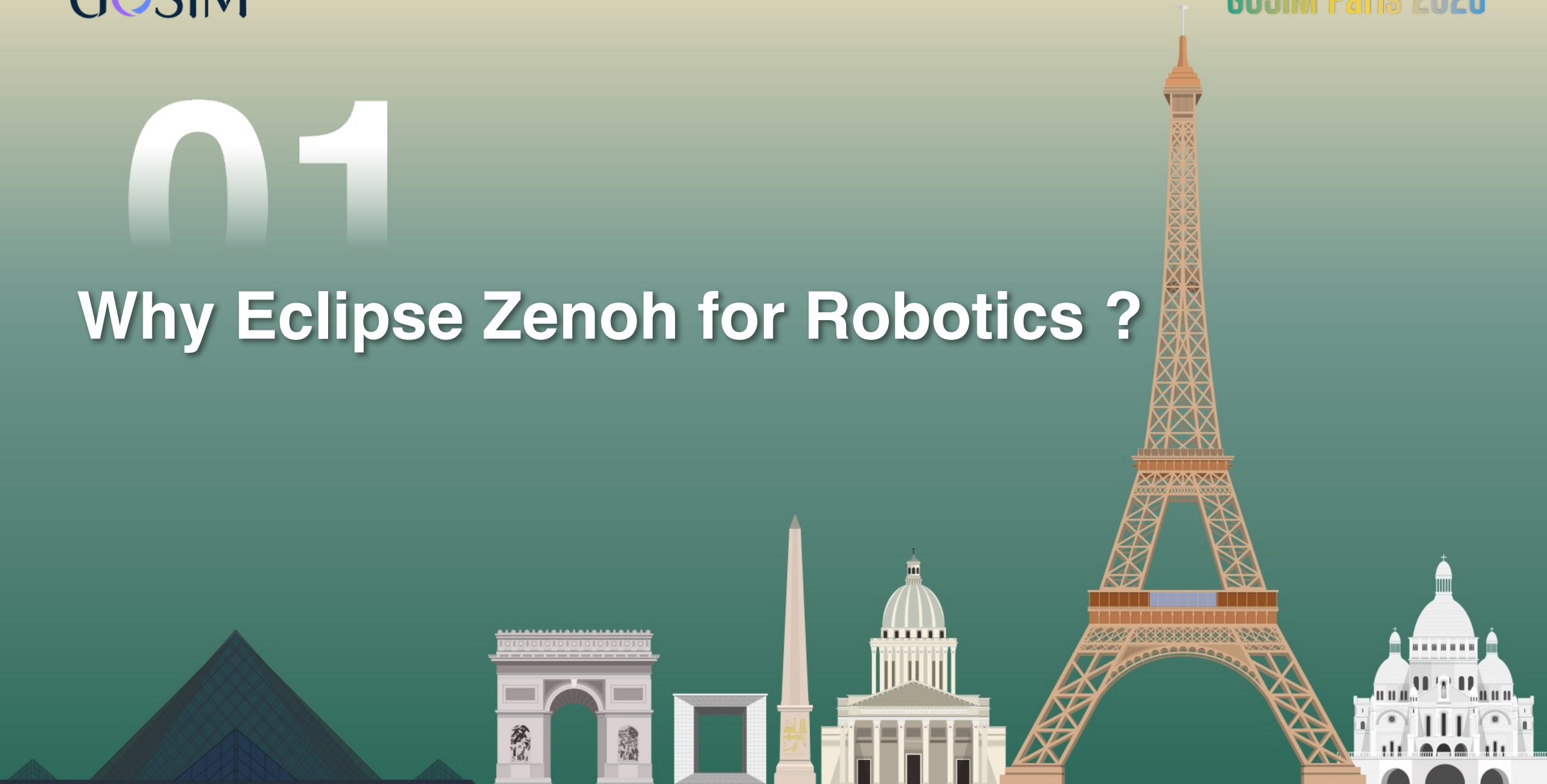
zetta scale

Julien Enoch

- Senior Solution Architect at ZettaScale Technology
- 25 Years of experience in distributed systems for Defence, Aerospace and Industry
- Eclipse Zenoh Committer
- rmw_zenoh maintainer
- Speaker at ROSCon, EclipseCon, FOSDEM...

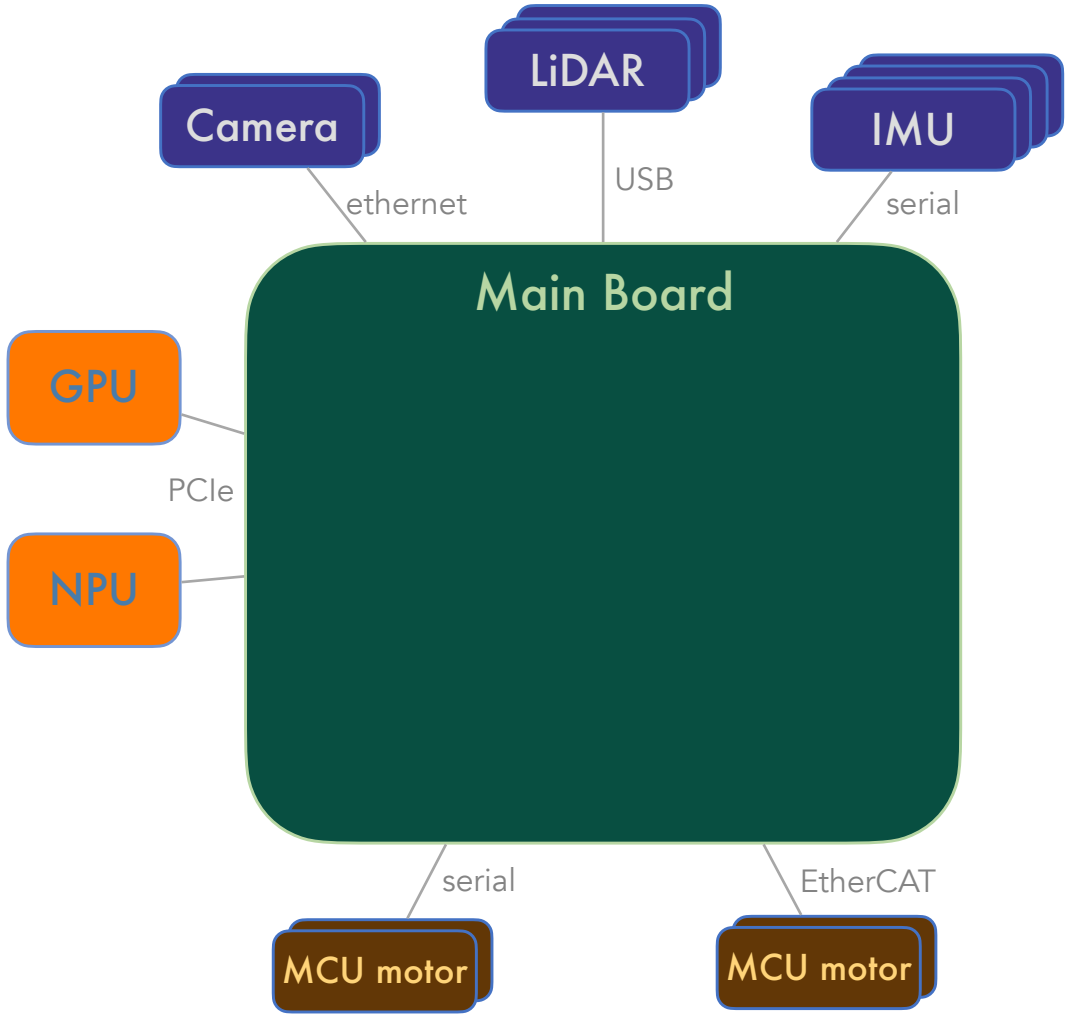
01

Why Eclipse Zenoh for Robotics ?



On-board

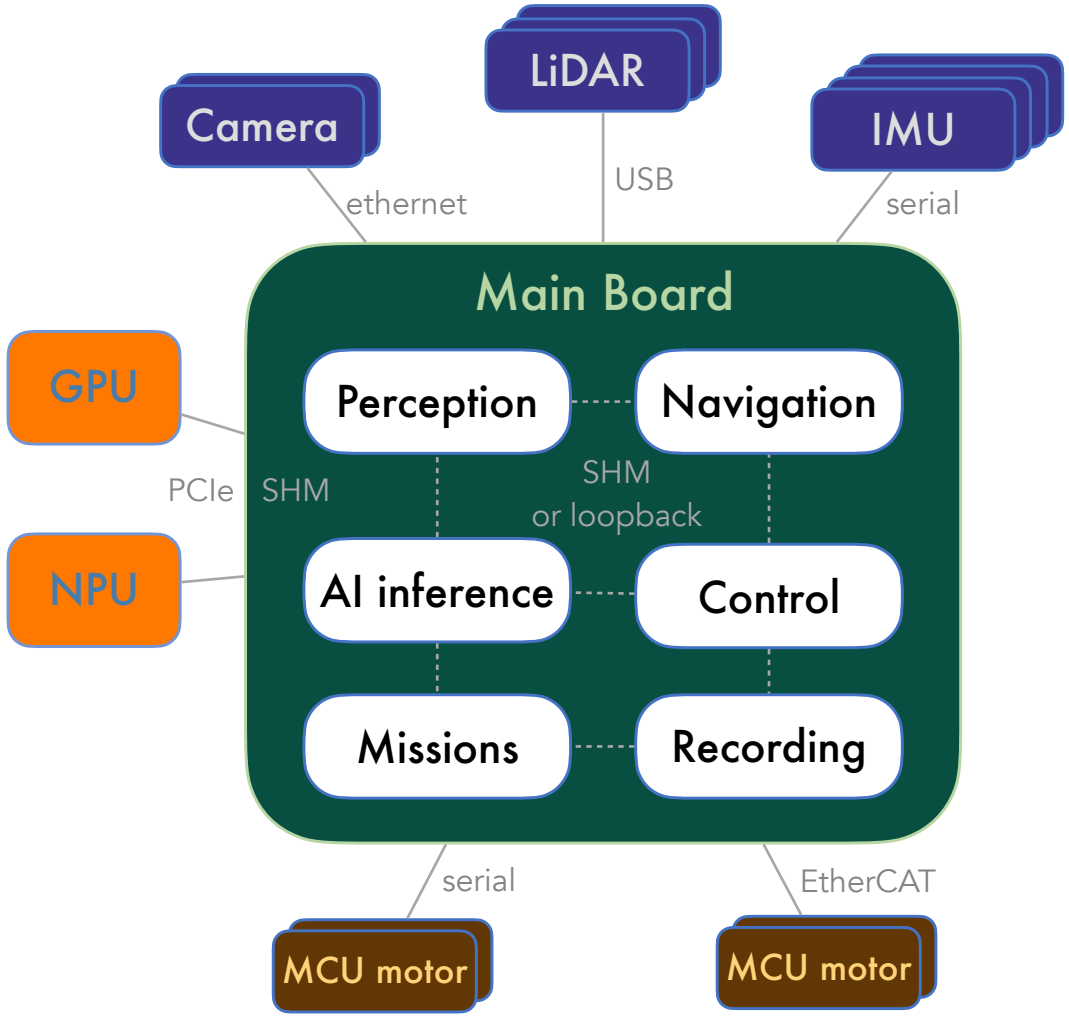
- Increasing complexity
- Multiple sensors, MCUs, GPUs, NPUs
- Heterogeneous transports:
  Serial, EtherCAT, Ethernet, SHM



On-board

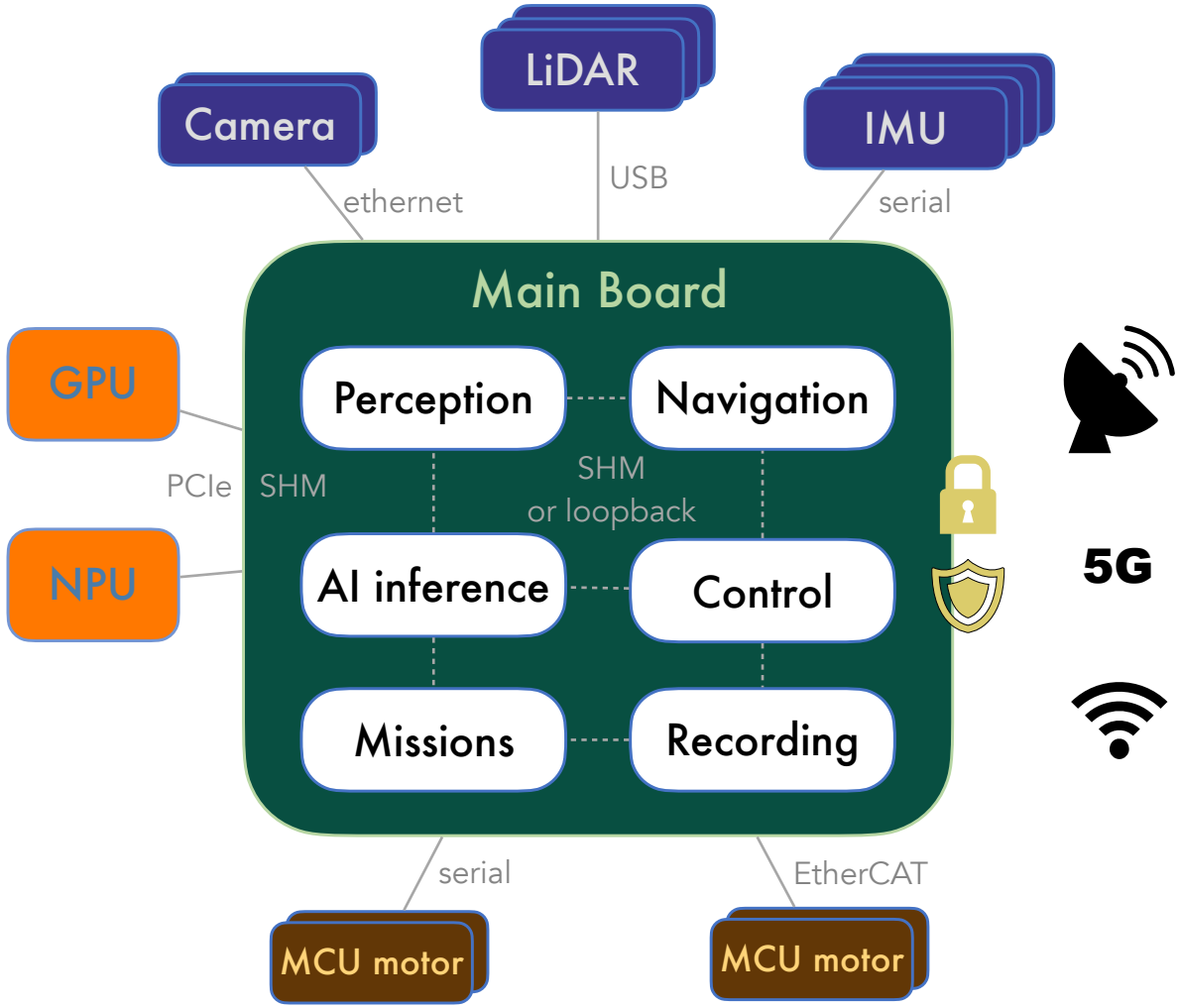
- Increasing complexity
- Multiple sensors, MCUs, GPUs, NPUs
- Heterogeneous transports:
  Serial, EtherCAT, Ethernet, SHM

- Flexible, modular architecture
- Independent components
- Peer-to-peer preferred

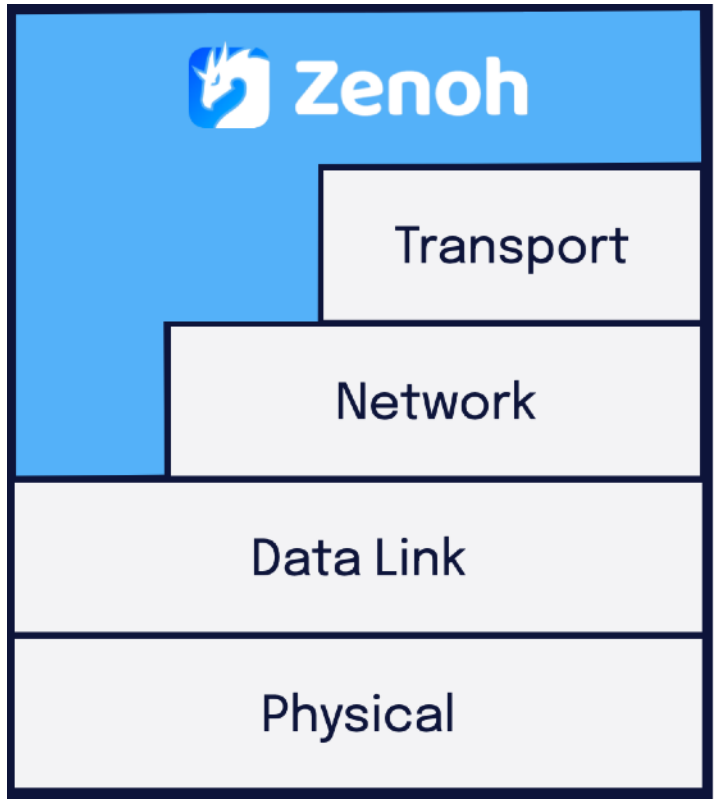


Off-board

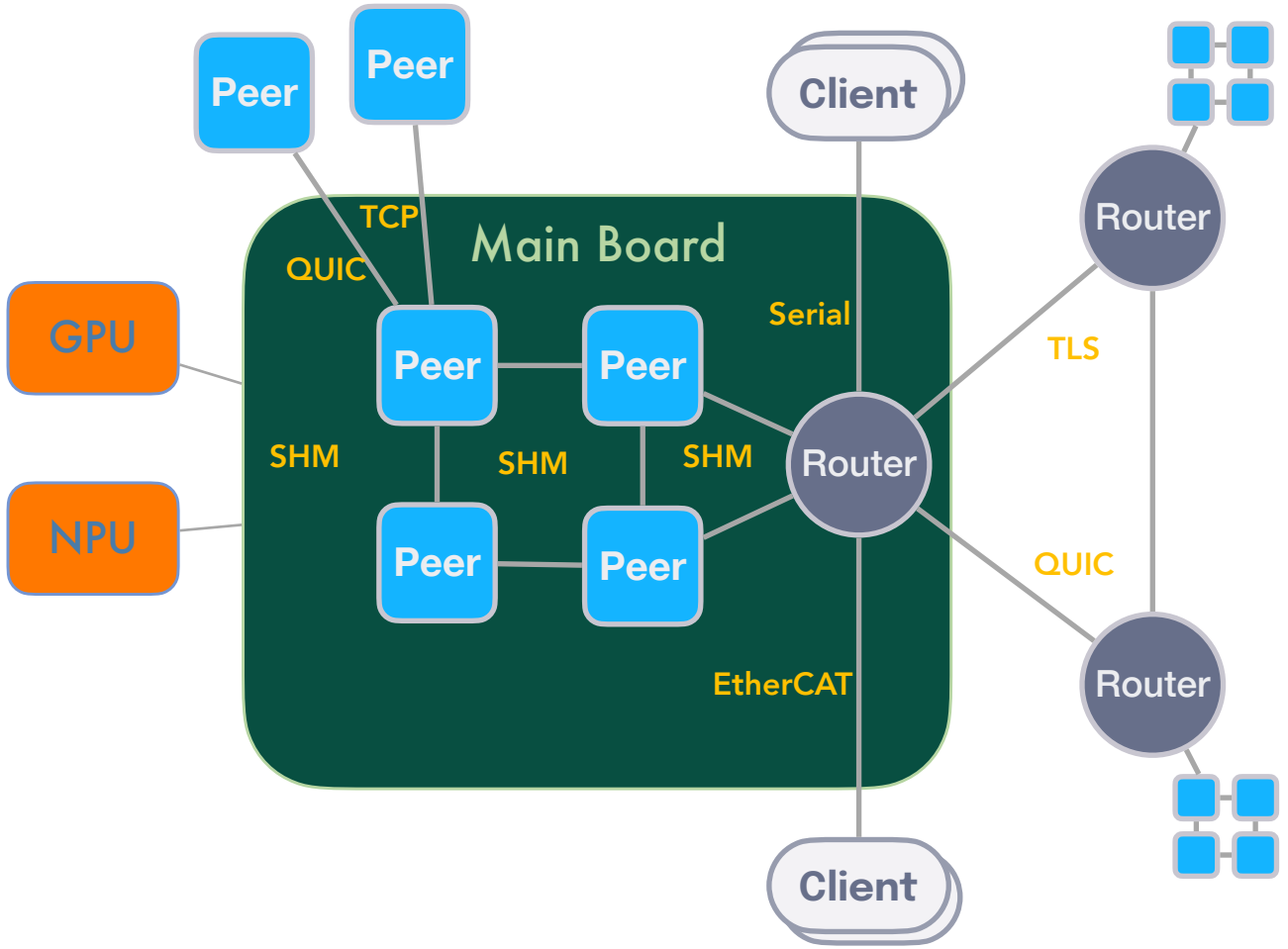
- Unreliable links: WiFi, 5G, satellite
- Limited bandwidth, packet loss, disconnections
- Security required: authentication, encryption, access control



- Supports network technologies from **transport** layer down-to the **data link**
- Currently implemented:
  - **TCP** with mTLS
  - **QUIC** with streams per-priority
  - **Serial**
  - UDP
  - WebSocket
  - UnixSocket

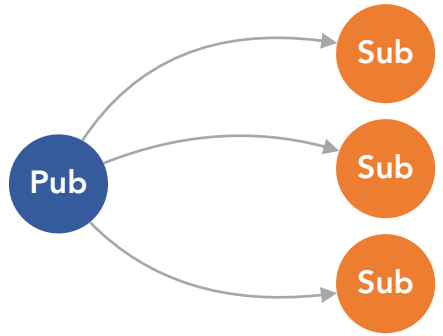


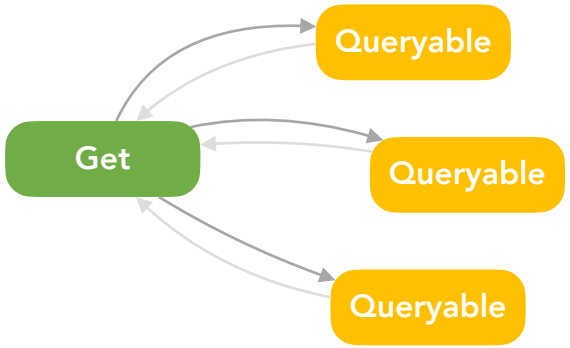
- Peer-to-peer and routed communications
- Multi-transports
- Shared Memory with zero-copy
- Zenoh Pico for MCUs



Communication patterns

- **Pub/Sub**
  For continuous data flows - decoupling, fan-out
- **Get/Queryable**
  For punctual interactions - commands, missions
- **Storages**
  For recording, blackboard or config retrieval
- **Liveliness Tokens**
  For discovery, supervision and reaction to failures





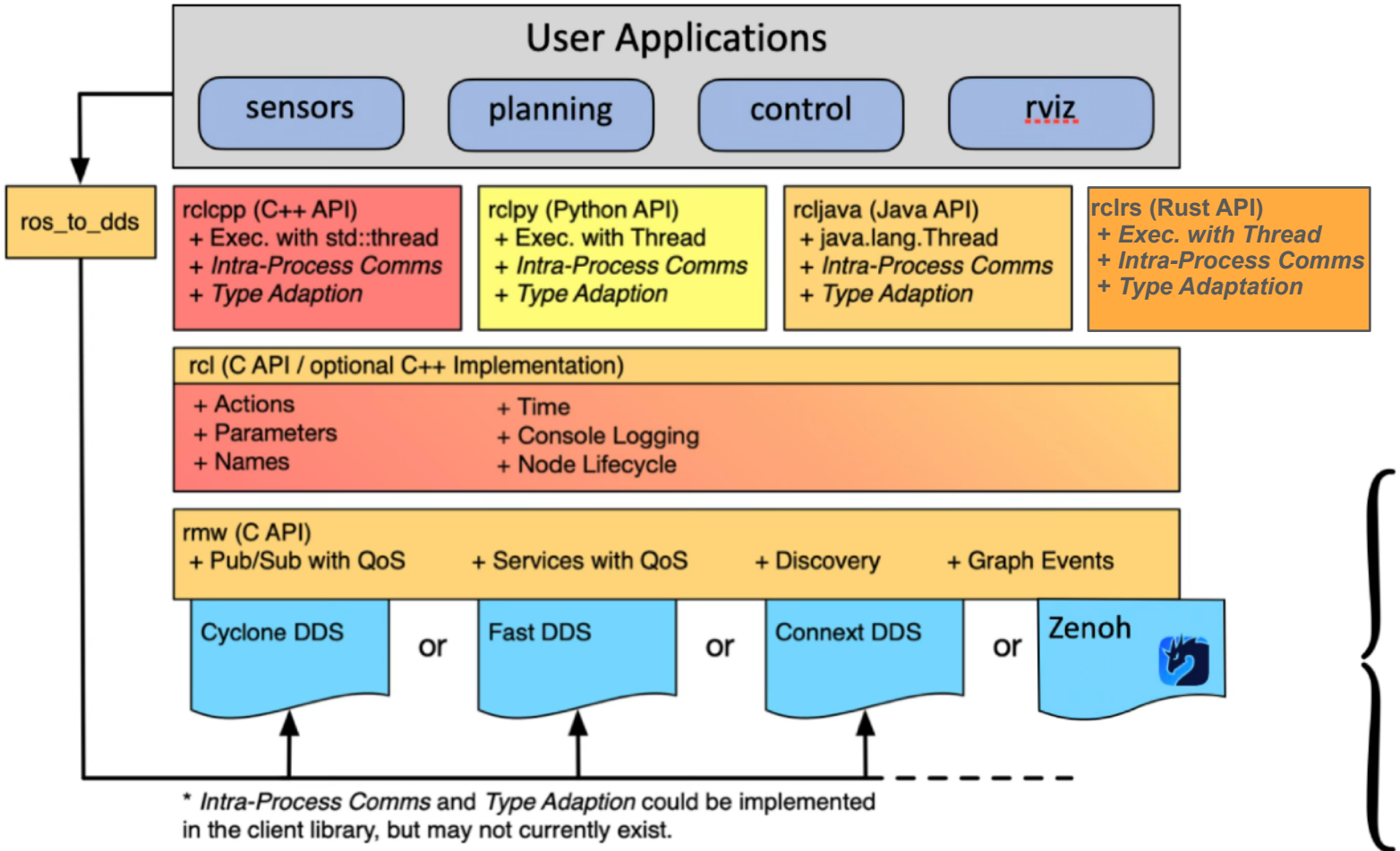
GOSIM

GOSIM Paris 2026

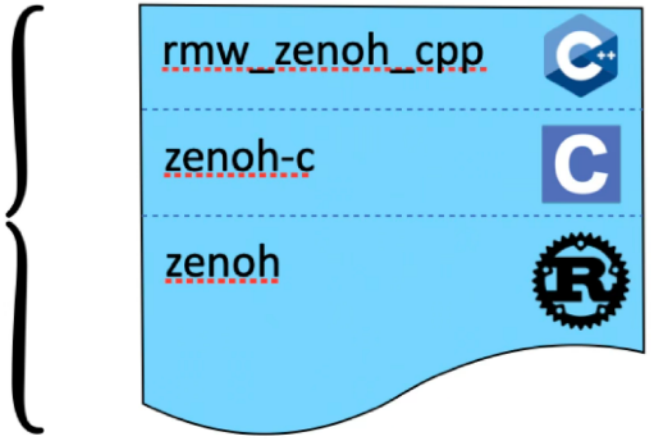
02

ROS & Zenoh

- **2022**
  - **Zenoh / DDS bridge**
    Reduces discovery traffic, solves WiFi issues
- **2023**
  - **Zenoh / ROS 2 bridge**
    Further traffic reduction, ROS graph support
- **2025**
  - **RMW Zenoh**
    Tier 1 for Kilted Kaiju



https://github.com/ros2/rmw_zenoh



https://docs.ros.org/en/rolling/Concepts/Advanced/About-Internal-Interfaces.html#internal-api-architecture-overview

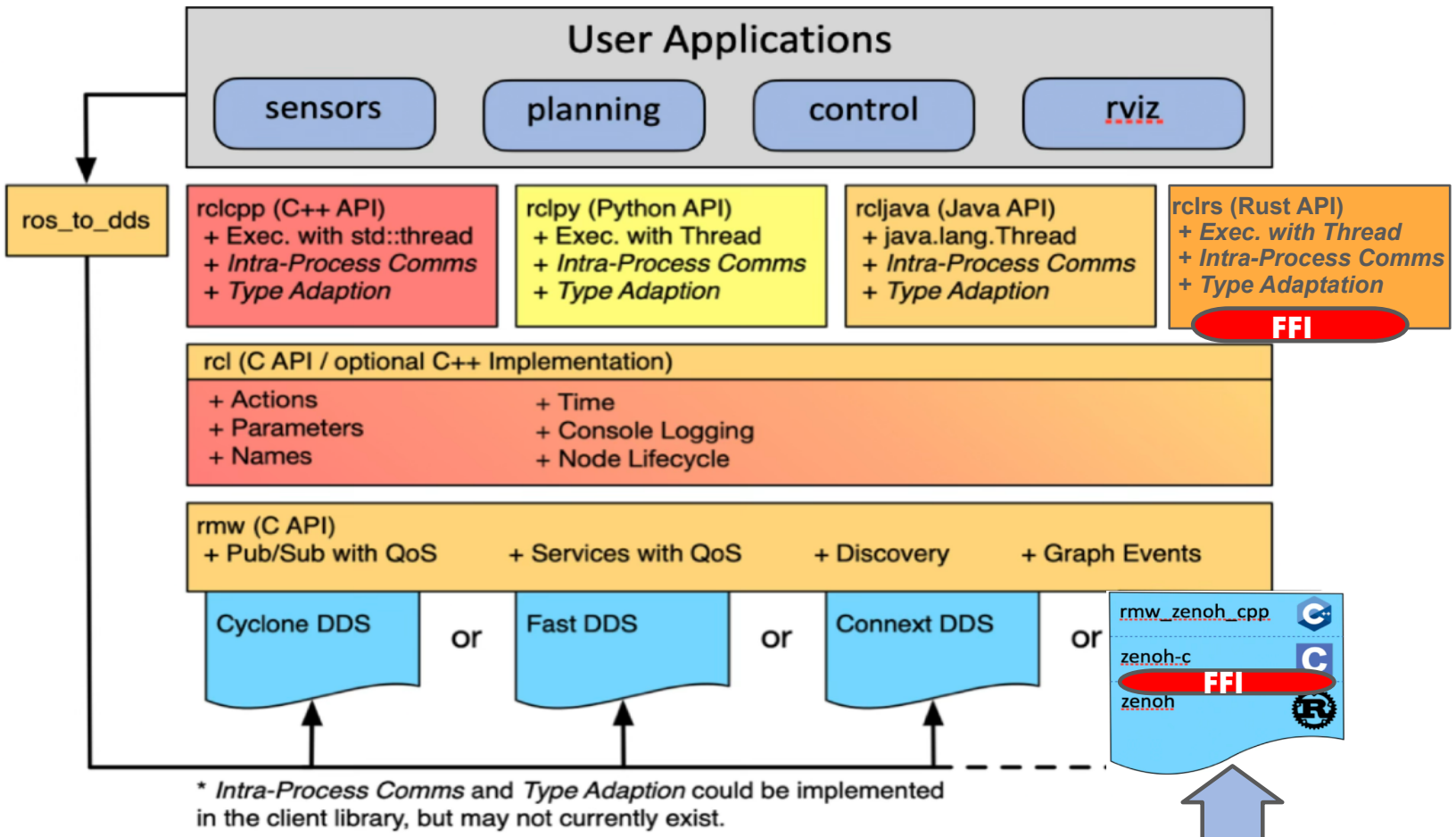
GOSIM

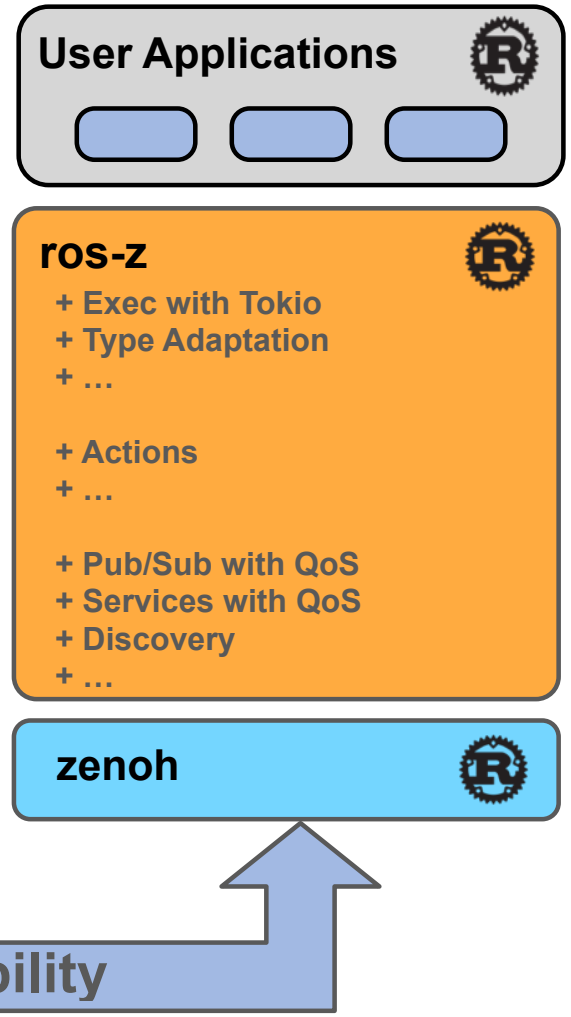
GOSIM Paris 2026

03

ROS-Z

What about a pure Rust stack ?





https://github.com/ZettaScaleLabs/ros-z

- **100% Rust**
  - Builder patterns for evolutivity with backward compatibility
  - Cargo build
- **Rust-native ROS 2 abstractions**
  - Pub/Sub, Services, Actions
  - Graph discovery
- **Battery include: Zero ROS 2 dependencies required**
  - Universal compatibility across Rust platforms
- **Interoperability with RMW Zenoh**
  - But also supporting alternative encodings (protobuf…)

Ergonomic API Design

ros-z provides flexible, idiomatic Rust APIs that adapt to your preferred programming style:

Flexible Builder Pattern:

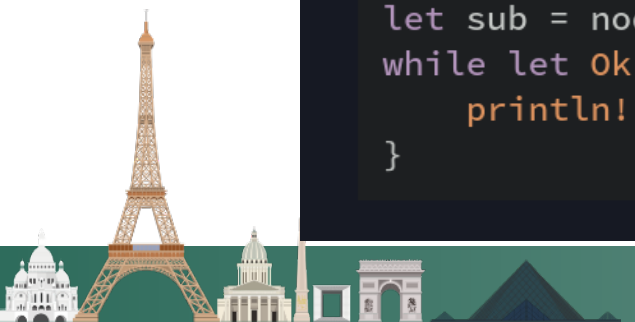
```
let pub = node.create_pub::<Vector3>("vector")
    // Quality of Service settings
    .with_qos(QosProfile {
        reliability: QosReliability::Reliable,
        ..Default::default()
    })
    // custom serialization
    .with_serdes::<ProtobufSerdes<Vector3>>()
    .build()?;
```

Async & Sync Patterns:

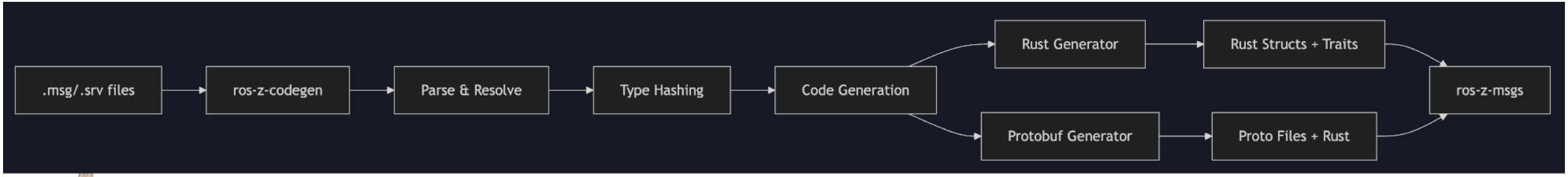
```rust
// Publishers: sync and async variants
zpub.publish(&msg)?;
zpub.async_publish(&msg).await?;

// Subscribers: sync and async receiving
let msg = zsub.recv()?;
let msg = zsub.async_recv().await?;
```

Callback or Polling Style for Subscribers:

```rust
// Callback style - process messages with a closure
let sub = node.create_sub::<RosString>("topic")
    .build_with_callback(|msg| {
        println!("Received: {}", msg);
    })?;

// Polling style - receive messages on demand
let sub = node.create_sub::<RosString>("topic").build()?;
while let Ok(msg) = sub.recv() {
    println!("Received: {}", msg);
}
```



- **New code generator in Rust**
  - Independent from ROS environment
  - Includes common ROS types
  - Parse .msg/.srv files
  - Multiple encoding: CDR or Protobuf



Mixed memory message is possible

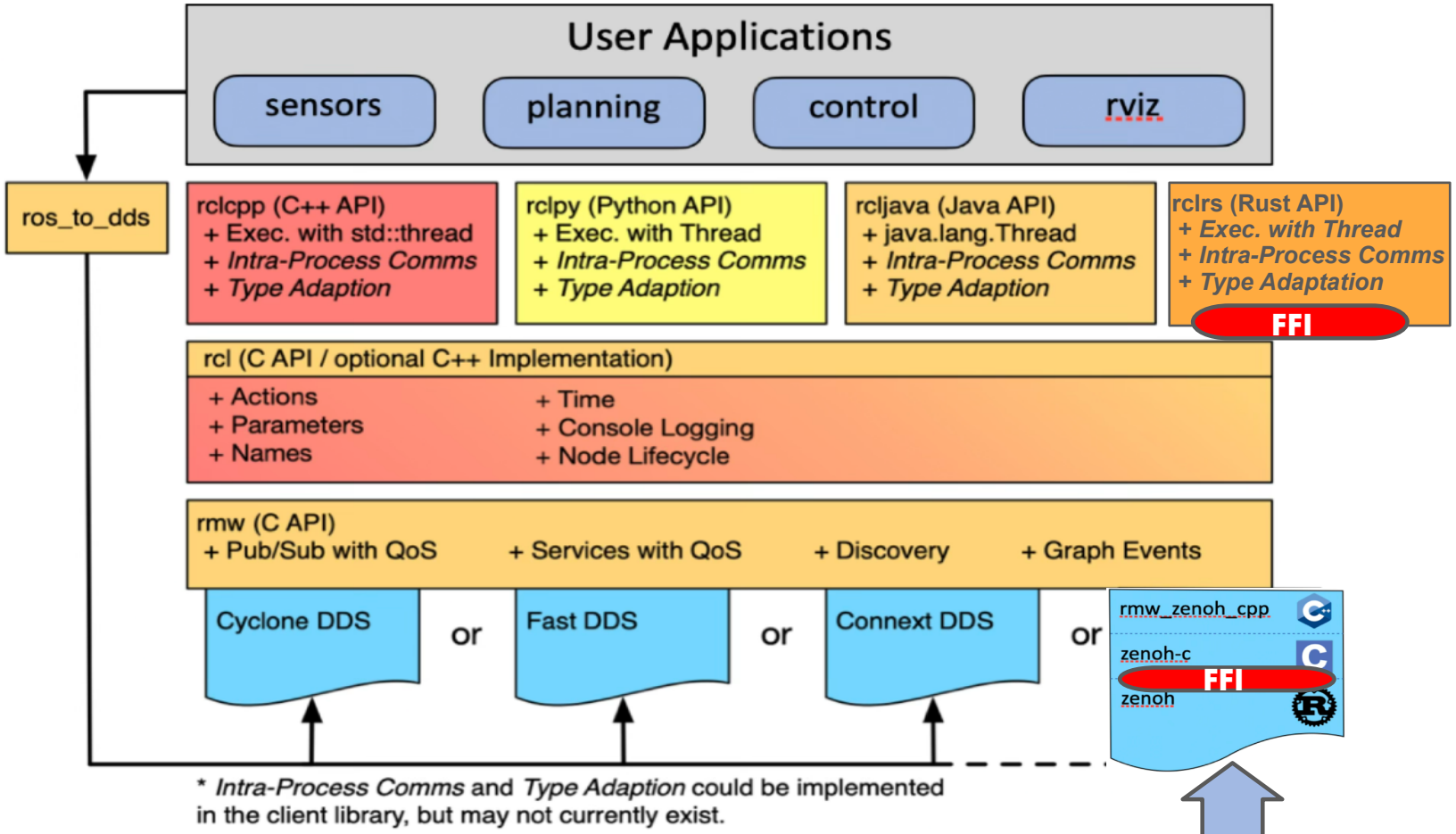
File: `sensor_msgs/PointCloud2.msg`

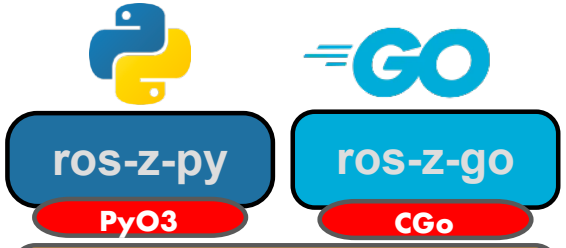
Raw Message Definition

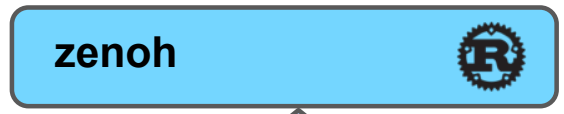
```
# This message holds a collection of N-dimensional points, which may
# contain additional information such as normals, intensity, etc. The
# point data is stored as a binary blob, its layout described by the
# contents of the "fields" array.

# The point cloud data may be organized 2d (image-like) or 1d
# (unordered). Point clouds organized as 2d images may be produced by
# camera depth sensors such as stereo or time-of-flight.

# Time of sensor data acquisition, and the coordinate frame ID (for 3d
# points).
Header header

# 2D structure of the point cloud. If the cloud is unordered, height is
# 1 and width is the length of the point cloud.
uint32 height
uint32 width

# Describes the channels and their layout in the binary data blob.
PointField[] fields

bool    is_bigendian # Is this data bigendian?
uint32  point_step   # Length of a point in bytes
uint32  row_step     # Length of a row in bytes
uint8[] data         # Actual point data, size is (row_step*height)

bool is_dense        # True if there are no invalid points
```

```rust
let point_step = 12; // x, y, z as f32 (4 bytes each)
let data_size = num_points * point_step;

// Allocate SHM buffer for point data
let mut shm_buf = provider
    .alloc(data_size)
    .with_policy::<BlockOn<GarbageCollect>>()
    .wait()?;

// Write point coordinates directly into SHM buffer

// Create ZBuf from SHM buffer (zero-copy conversion!)
let data_zbuf = ZBuf::from(shm_buf);

// Construct PointCloud2 with SHM-backed ZBuf
Ok(PointCloud2 {
    header: Header {
        frame_id: "map".into(),
        ..Default::default()
    },
    height: 1,
    width: num_points as u32,
    fields: ...,
    is_bigendian: false,
    point_step: point_step as u32,
    row_step: (num_points * point_step) as u32,
    data: data_zbuf, // SHM-backed data!
    is_dense: true,
})
```

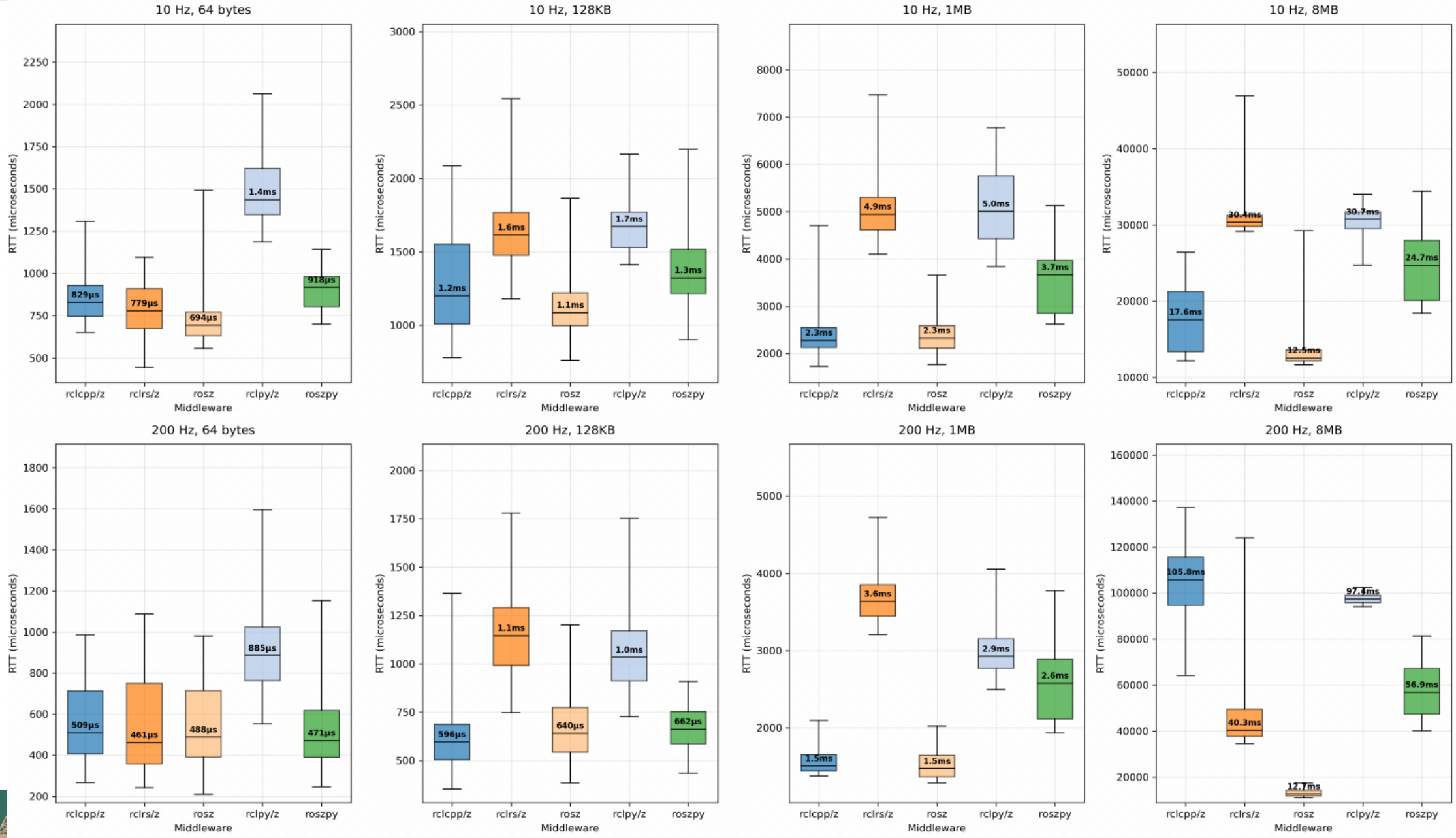
Other languages bindings







Latency


10 Hz, 64 bytes
RTT (microseconds)
Middleware
rclcpp/z
rclrs/z
rosz
rclpy/z
roszpy
829µs
779µs
694µs
1.4ms
918µs
10 Hz, 128KB
1.2ms
1.6ms
1.1ms
1.7ms
1.3ms
10 Hz, 1MB
2.3ms
4.9ms
2.3ms
5.0ms
3.7ms
10 Hz, 8MB
17.6ms
30.4ms
12.5ms
30.7ms
24.7ms
200 Hz, 64 bytes
509µs
461µs
488µs
885µs
471µs
200 Hz, 128KB
596µs
1.1ms
640µs
1.0ms
662µs
200 Hz, 1MB
1.5ms
3.6ms
1.5ms
2.9ms
2.6ms
200 Hz, 8MB
105.8ms
40.3ms
12.7ms
97.4ms
56.9ms

04

Conclusion

ROS

- **Zenoh is an official middleware for ROS 2**
  rmw_zenoh is Tier 1, maintained by Open Robotics & ZettaScale
- **A growing ecosystem beyond ROS**

dora-rs

copper-rs

pico ROS

OPENMIND

Cerulion

- **ros-z - one step further**

Dexory warehouse robot using rmw_zenoh

Supports ROS 2

- Interoperable with ROS 2
- 100% Rust, no ROS environment required
- Other language bindings (Python, Go…)

Enhances ROS 2

- Alternative encodings (Protobuf…)
- Mixed memory messages
- Lower latency
- Easiest way to bridge ROS 2 with Zenoh-based framework

European projects supporting ZettaScale for Zenoh development:

Trustworthy, Cognitive and AI-Driven Collaborative associations of IoT devices and edge resources for data processing

EMPYREAN research project has received funding from the European Union's HORIZON Europe under the Grant Agreement n° 101136024.

Intelligent, Safe & secure connected Electrical Mobility solutions: Towards European Green Deal & Seamless Mobility

EcoMobility has received funding from Chips Joint Undertaking (Chips JU) under Grant Agreement No 101096387. Co-funded by European Union.

Enabling safe & secure modular UPdates, UPgrades and DynAmic Task reallocation and Execution for Software-Defined Vehicles

EMPYREAN research project has received funding from the European Union's HORIZON Europe under the Grant Agreement n° No. 101203060.

Decentralized Edge Intelligence: Advancing Trust, Safety, and Sustainability in Europe

EdgeAI-Trust "Decentralized Edge Intelligence: Advancing Trust, Safety, and Sustainability in Europe" project has received funding from Chips Joint Undertaking (Chips JU) under Grant Agreement No 101139892-2.

O-CEI Open CloudEdgeIoT Platform Uptake in Large Scale Cross-Domain Pilots

O-CEI project has received funding from the European Union's Horizon Europe Framework Programme under the Grant Agreement N° 101189589.

Funded by the European Union

GOSIM

GOSIM Paris 2026

Thanks!